

EcoHarmony Park - App

Hoja de estilos



Integrantes:

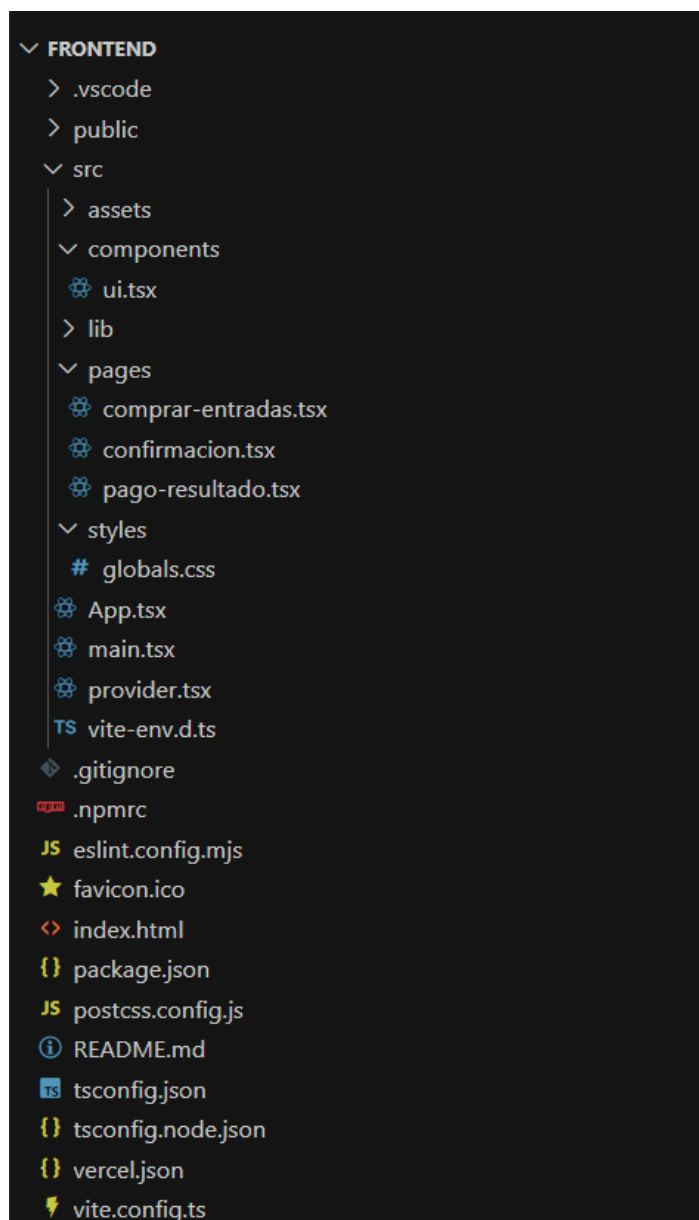
- Bobroviec, Ivan (95909)
- Cavilla, Matias (90589)
- Costanzo, Franco (89478)
- Höhlke, Augusto (90263)
- Romero Justo, Milton Jesus (68396)
- Sosa, Diego (82397)

1. Arquitectura y convenciones Frontend

1.1 Stack Tecnológico Principal

- **Framework Base:** React (v19.0) con TypeScript (`.tsx`) para un tipado estático seguro
- **Entorno de Construcción:** Vite (v6.0) como empaquetador moderno y de alta velocidad.
- **Estilos y Diseño:** Tailwind CSS (v4.0) acoplado a la librería de componentes HeroUI (v3.1) para interfaces limpias y adaptativas.
- **Enrutamiento:** React Router Dom (v6.23) para la gestión declarativa de vistas de la aplicación.

1.2 Organización y Estructura de Directorios



1.3 Reglas de Estilos y Paleta de Colores

Queda estrictamente prohibido el uso indiscriminado de estilos en línea en el HTML que alteren la identidad de marca. El diseño se gobierna enteramente mediante clases utilitarias de Tailwind aplicadas directamente en los elementos y tokens CSS definidos en `globals.css`:

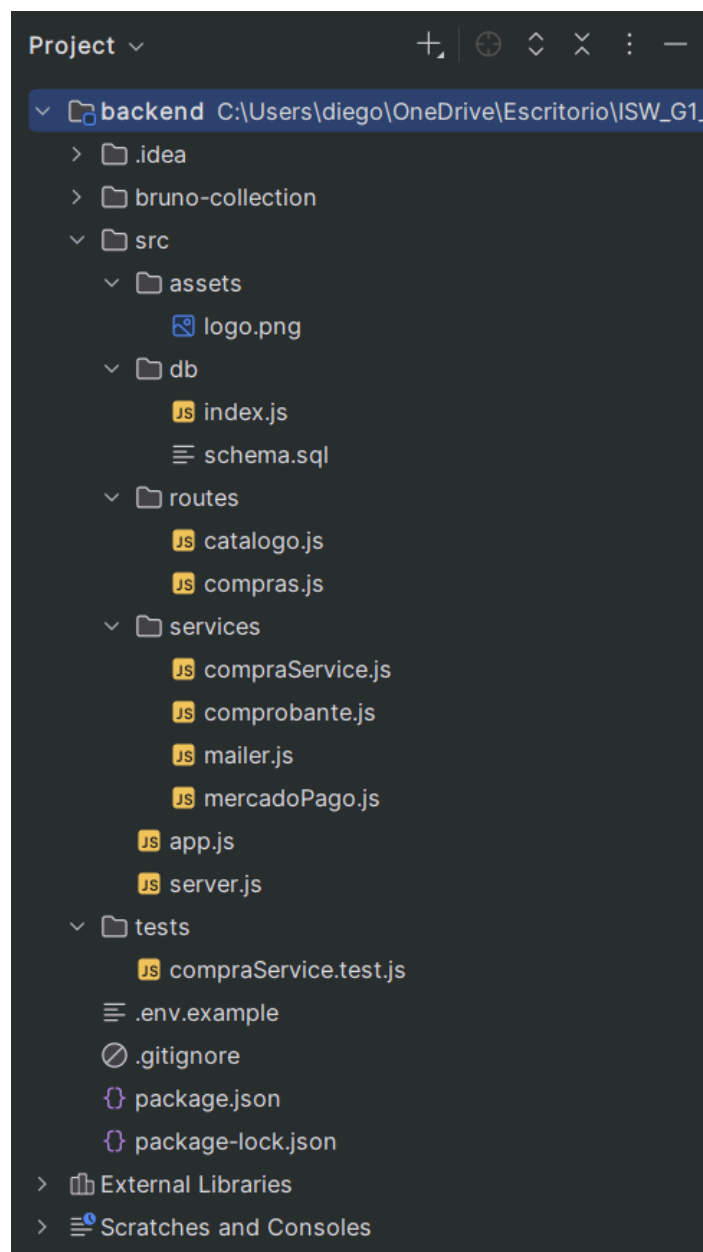
- **Tipografía Oficial:** `"Lato", sans-serif` para textos legibles y visuales; `"JetBrains Mono", monospace` para datos técnicos (códigos QR).
- **Hunter Green (--color-p6: #386641):** Verde oscuro para cabeceras prominentes, gradientes de éxito y textos primarios del parque.
- **Sage Green (--color-p5: #6a994e):** Color de acento principal utilizado en botones, selecciones de fechas y estados confirmados.
- **Yellow Green (--color-a5: #a7c957):** Acento vivo para indicadores secundarios y complementos de gradientes.
- **Champagne Mist (--color-champagne: #f2e8cf):** Relleno cálido de fondos de fondo de contenedor y marcos.
- **Blushed Brick (--color-brick: #bc4749):** Color de alerta semántica aplicado exclusivamente a mensajes de error y rechazos de pago.

2. Arquitectura y convenciones Backend

2.1 Stack Tecnológico Principal

- **Entorno de Ejecución:** Node.js con el framework Express para el ruteo de la API REST
- **Mecanismo de Persistencia:** Base de datos relacional integrada SQLite3 gestionada con la librería de alto rendimiento `better-sqlite3`.
- **Librería de Soporte Críticas:** `pdfkit` (para renderizar comprobantes oficiales en PDF), `qrcode` (para codificar los identificadores únicos UUID de los pases) y `nodemailer` (para el despacho de correos electrónicos).

2.2 Organización y Estructura de Directorios



3. Estándares Transversales de código y Nomenclatura

3.1 Convenciones de Nombres

Para asegurar que el código sea autoexplicativo, se fijan las siguientes pautas estrictas de nombrado:

Elemento de Código	Convención Aplicada	Ejemplo de Implementacion Real
Componentes de React	PascalCase	ComprarEntradas(), IconoResultado(), Navbar()
Archivos de Código (.ts, .tsx, .js)	camelCase o kebab-case	compraService.js, pago-resultado.tsx, api.ts
Variables y Funciones Lógicas	camelCase	montoTotal, buscarEstadoPorReferencia(), fechaVisita
Clases e Interfaces (TS)	PascalCase	interface TipoTicket, class CompraService
Constantes Globales y Entorno	SNAKE_CASE (Mayúsculas)	JWT_SECRET, MAX_REINTENTOS, FERIADOS, PORT
Tablas de Base de Datos (SQL)	PascalCase (Plural)	Usuarios, Horarios, Compras, Tickets, TiposTicket

3.2 Estructura de funciones y Buenas Prácticas

- **Uso de Funciones Flecha (Arrow Functions):** Se prioriza su uso (`const miFuncion = () => {}`) tanto en el front como en el desarrollo de controladores del back para mantener contextos limpios y compactos.
- **Estrategia Asíncronica Unificada:** Toda operación que involucre llamadas de red (API fetch) o consultas a la base de datos de SQLite debe implementar rigurosamente la estructura `async/await`. Queda desaconsejado el encadenamiento excesivo de promesas estructuradas con `.then()` en el backend para evitar problemas de legibilidad.
- **Declaración Unitaria de Variables:** Se prohíbe agrupar variables en una sola línea separadas por comas. Cada variable mutante (`let`) o inmutable (`const`) debe declararse de forma individual en su propia línea física de código para no comprometer el rastreo en auditorías de cambios.

4. Control de Errores y Comentarios

4.1 Robustez y Manejo de Excepciones

El sistema debe ser tolerante a fallos y jamás exponer caídas críticas al cliente. Los bloques `try-catch` deben implementarse de manera mandatoria en todos los controladores de rutas del backend y en los despachos asíncronos del frontend.

```
function comprasRoutes({ service }) {
  const router = express.Router();

  router.post('/', async (req, res) => {
    try {
      const { fechaVisita, metodoPago, tickets } = req.body || {};
      const result = await service.comprar({
        usuarioId: HARDCODED_USER.id,
        fechaVisita,
        metodoPago,
        tickets,
      });
      res.status(201).json(result);
    } catch (e) {
      if (e instanceof CompraError) return res.status(e.status).json({ error: e.message });
      console.error(e);
      res.status(500).json({ error: 'Error interno' });
    }
  });
}
```

4.2 Comentarios y Calidad de Documentación

- Los comentarios deben ser utilizados con prudencia y moderación, **únicamente** cuando la lógica de negocio subyacente sea intrínsecamente compleja o responda a una restricción externa no evidente (por ejemplo, el control de zonas horarias locales al parsear cadenas de texto ISO).
- **Formato permitido:** con `//` para comentarios explicativos de una sola línea de código; y bloques cerrados `/* ... */` para comentarios multilínea o notas extensas de arquitectura en las cabeceras de los módulos de servicios.